Module 1 — Relational data and SQL foundations

Learning outcomes

By the end, you should be able to describe a relational table, identify its grain, define keys and constraints, write simple SELECT queries, filter and sort rows, and explain NULL.

1. Tables, rows, columns and grain

A table stores records with a consistent set of attributes. A row is one record. The **grain** states what one row represents. Example: in `order_items`, one row may represent one product line within one order—not one whole order.

Always identify grain before joining or aggregating. If an order has three item rows, joining the order to items produces three rows for that order.

2. Keys

• Primary key: chosen unique, non-null identifier.

• Candidate key: minimal set of columns that can uniquely identify a row.

• Foreign key: references a key in another table.

• Composite key: uses multiple columns.

• Natural key: business meaning (email, product code); surrogate key: generated identifier.

A key is not merely a column called `id`; uniqueness must be true in the data.

3. Constraints

`NOT NULL`, `UNIQUE`, `CHECK`, `DEFAULT`, `PRIMARY KEY`, `FOREIGN KEY`.

Constraints protect data even when multiple applications write to the database. Application validation alone is not enough.

4. Basic query shape

```
SELECT customer_id, city
FROM customers
WHERE city = 'Mysuru'
ORDER BY customer_id
LIMIT 10;
```

• SELECT chooses output columns/expressions.

• FROM identifies source.

• WHERE filters rows.

• ORDER BY sorts.

• LIMIT restricts returned rows (syntax differs by DBMS).

5. Operators and filtering

Comparison: =, <>, >, >=, <, <=.

Logical: `AND`, `OR`, `NOT`.

Range: `BETWEEN` (inclusive endpoints).

Set membership: `IN`.

Pattern: `LIKE` (% any sequence, _ one character).

NULL: `IS NULL`, `IS NOT NULL`.

6. NULL is not zero or empty text

NULL means missing/unknown/not applicable. `NULL = NULL` is not TRUE; it is UNKNOWN. Use `IS NULL`. Aggregates such as `SUM(column)` ignore NULL values; `COUNT(*)` counts rows and `COUNT(column)` counts non-NULL values.

Worked example

```
SELECT product_id,
       quantity * unit_price AS line_value
FROM order_items
WHERE quantity >= 2
ORDER BY line_value DESC;
```

If either quantity or price is NULL, `line_value` is NULL. Decide whether such rows should be excluded, flagged or treated as invalid; do not silently replace missing prices with zero without a business rule.

Guided exercises

Use:

```
customers(customer_id, customer_name, city, signup_date)
products(product_id, product_name, category, list_price, active)
```

1. Return all customers from Bengaluru.

2. Return products with list price between 500 and 1500.

3. Find inactive products.

4. Return customers whose city is missing.

5. Return the 5 most recently registered customers.

6. Find product names containing the word `phone`.

7. Return distinct cities, including the NULL city if present.

8. Return products where price is missing or non-positive.

Solutions

```
-- 1
SELECT * FROM customers WHERE city = 'Bengaluru';

-- 2
SELECT * FROM products WHERE list_price BETWEEN 500 AND 1500;

-- 3
SELECT * FROM products WHERE active = FALSE;

-- 4
SELECT * FROM customers WHERE city IS NULL;

-- 5
SELECT * FROM customers
ORDER BY signup_date DESC, customer_id DESC
LIMIT 5;

-- 6
SELECT * FROM products WHERE product_name ILIKE '%phone%';

-- 7
SELECT DISTINCT city FROM customers;

-- 8
SELECT * FROM products
WHERE list_price IS NULL OR list_price <= 0;
```

`ILIKE` is PostgreSQL-specific. In other systems use an appropriate case-insensitive comparison.

Independent exercises

9. Find customers who signed up during 2025 using a half-open date range.

10. Return products whose category is either `Electronics` or `Home`.

11. Sort products by category ascending and price descending, with NULL prices last.

12. Find duplicate customer emails in a hypothetical table `customer_contacts(customer_id, email)`.

Solutions

```
-- 9
SELECT * FROM customers
WHERE signup_date >= DATE '2025-01-01'
  AND signup_date <  DATE '2026-01-01';

-- 10
SELECT * FROM products
WHERE category IN ('Electronics', 'Home');

-- 11
SELECT * FROM products
ORDER BY category ASC, list_price DESC NULLS LAST;

-- 12
SELECT email, COUNT(*) AS occurrences
```

```
FROM customer_contacts
WHERE email IS NOT NULL
GROUP BY email
HAVING COUNT(*) > 1;
```

Concept test

1. Can a primary key contain NULL? No.

2. Does `COUNT(column)` count NULL? No.

3. Is NULL the same as zero? No.

4. Why is grain important? It determines what a row means and prevents incorrect joins/aggregates.

5. Is a foreign key automatically NOT NULL? No; add NOT NULL if every row must reference a parent.

Interview challenge

A table has 10,000 rows and 8,000 distinct order IDs. What could explain the difference? Possible answer: the table may be at order-line grain, with multiple rows per order. Do not assume the rows are duplicate transactions.

External practice for this module

Complete these in order, alongside the exercises above:

1. SQLBolt — interactive lessons — complete lessons 1–6 first for SELECT, filtering, sorting and basic joins. Then complete the remaining lessons as revision.

2. HackerRank SQL — Basic Select — solve at least 8 Basic Select problems, including filtering, sorting, pattern matching and NULL handling.

3. SQLZoo — SELECT basics and SELECT from World — solve the tutorial tasks without looking at answers.

4. LeetCode SQL 50 — complete the Select section. Record which questions required a second attempt.

Exit requirement: write 10 SELECT queries from a fresh schema without notes; explain `NULL`, `DISTINCT`, ordering and date boundaries. Use SQLite on SQLBolt/SQLZoo for fundamentals, then repeat representative queries in PostgreSQL.